PROJE RAPOR

AD / SOYAD : MEHMET SALİH TÜYSÜZ

NO : G231210017

E-posta : salih.tuysuz@ogr.sakarya.edu.tr

AD / SOYAD : ALPEREN SOMUNCUOĞLU

NO : G231210029

E-posta : alperen.somuncuoğlu@ogr.sakarya.edu.tr

Uygulamamız F1 ve yarışları hakkında; sürücü, araba, sponsor ,pist gibi farklı varlıkları ve niteliklerini içeriyor.

1. Bir kişi, disjoint kalıtım şekilde 3e sınıflandırılabilir: Takım Üyesi, Sürücü, Sponsor Yöneticisi.
2. Her takımda bir sürücü bulunur ve her bir sürücünün bir takımı bulunur.
3. Takım Üyesi, bir takıma ait olabilir ve herhangi bir takımın birden fazla üyesi olabilir.
4. Sponsor Yöneticisi, bir sponsora bağlıdır ve her sponsorun bir yöneticisi bulunur.
5. Her bir sponsorun bir il bilgisi vardır ve her ilde sıfır veya birden fazla sponsor olabilir.
6. Her sponsor bir takımı sponsor edebilir ve her takım birden fazla sponsora sahip olabilir.
7. Araba, bir sürücüye aittir ve her sürücünün bir arabası vardır .
8. Her araç malzemesi sadece bir arabaya aittir ve bir araba, bir veya birden fazla araç malzemesine sahip olabilir .
9. Bir malzeme sıfır veya birden fazla araç malzemesi olabilir, ancak araç malzemesi yalnızca bir malzeme türüdür.
10. Bir malzemenin bir sponsoru vardır, sponsorun 0 veya çok malzemesi olabilir.
11. Her takımın, sıfır veya birden fazla yarış ara bağlantısına sahip olması mümkündür ve her yarışta bir veya birden fazla takım yarışı ara bağlantısı bulunabilir.
12. Her ilde bir veya birden fazla ilçe olabilir ve herhangi bir ilçe bir ilde bulunur .
13. Her ilde bir pist vardır ve her pist bir ilde bulunur.
14. Her yarışın yalnızca 1 hava durumu vardır, bir hava durumu 0 veya çok yarışta olabilir.
15. Bir yarışta bir pist bulunabilir ve bir pist, 0 veya birden fazla yarışta yer alabilir.
16. Bir lastik türü sıfır veya birden fazla takımda kullanılır ve her takım bir lastik türü kullanır.
17. Her yarışta bir tane lastik türü önerilir, bir lastik türü 0 veya çok yarışta önerilebilir.

———İlişkisel Şema———

Araba(ArabaID: Integer, SurucuID: Integer)

AracMalzeme(AracMalzemeID: Integer, ArabaID: Integer, MalzemeID: Integer)

Malzeme(MalzemeID: Integer, SponsorID: Integer, MalzemeTuru: String)

LastikTur(LastikTurID: Integer, LastikTuru: String)

Ilce(IlceID: Integer, IlID: Integer, IlceAdi: String)

Il(IlID: Integer, IlAdi: String)

Sponsor(SponsorID: Integer, IlID: Integer, SponsorYoneticiID: Integer)

SponsorYonetici(SponsorYoneticiID: Integer, YoneticiAdi: String, YoneticiSoyadi: String)

Takim(TakimID: Integer, SponsorID: Integer)

Surucu(SurucuID: Integer, TakimID: Integer, ArabaID: Integer)

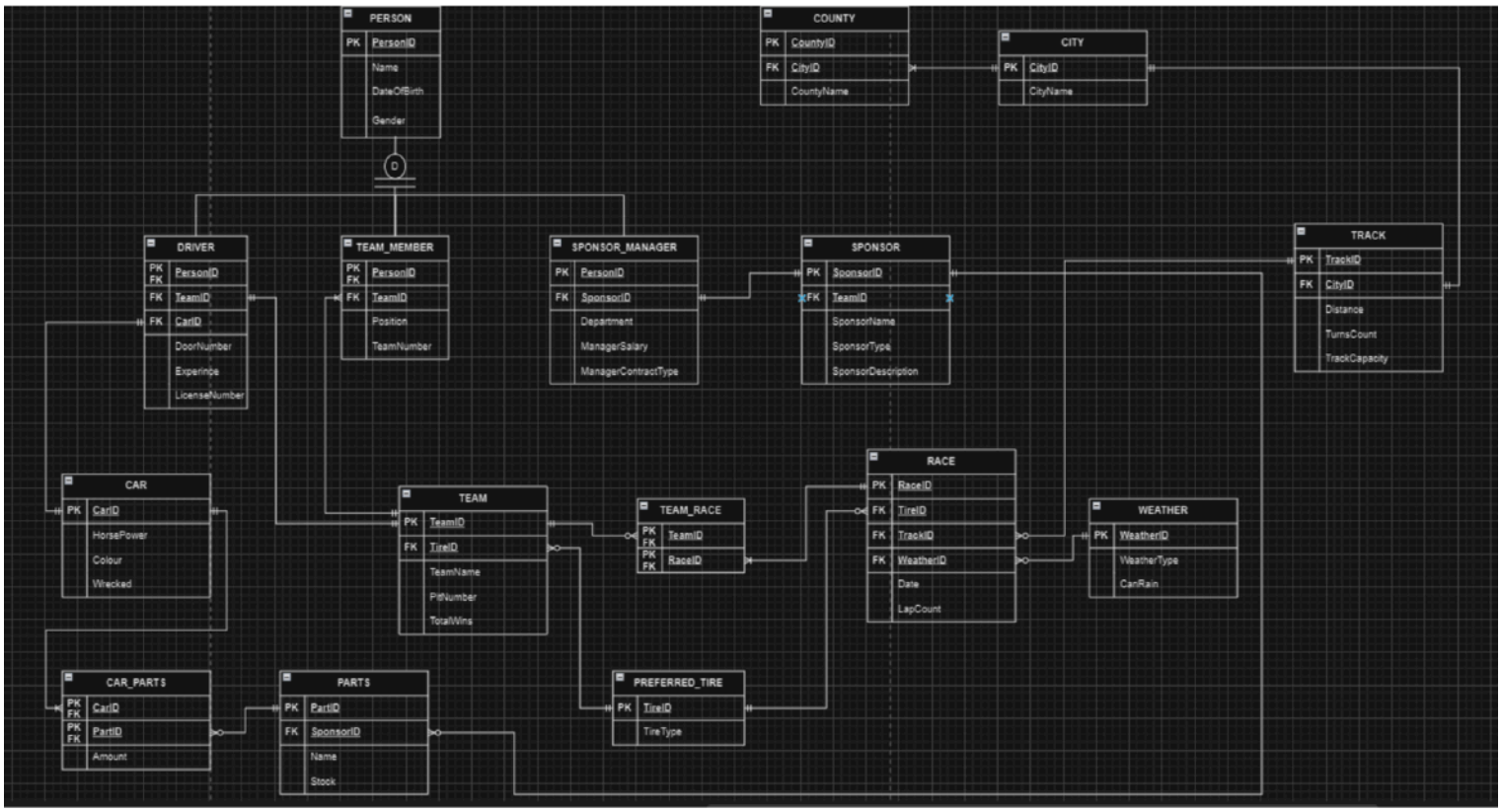
TakimUyesi(TakimUyesiID: Integer, TakimID: Integer)

Yaris(YarisID: Integer, IlID: Integer, PistID: Integer, HavaDurumuID: Integer, LastikTuruID: Integer)

Pist(PistID: Integer, IlID: Integer, PistAdi: String)

HavaDurumu(HavaDurumuID: Integer, HavaDurumu: String)

———Varlık Bağıntı modeli ———




```
CREATE TABLE Araba (
ArabaID INT PRIMARY KEY,
SurucuID INT,
FOREIGN KEY (SurucuID) REFERENCES Surucu(SurucuID)
);

CREATE TABLE AracMalzeme (
AracMalzemeID INT PRIMARY KEY,ArabaID INT,
MalzemeID INT,
FOREIGN KEY (ArabaID) REFERENCES Araba(ArabaID),
FOREIGN KEY (MalzemeID) REFERENCES Malzeme(MalzemeID)
);

CREATE TABLE Malzeme (
MalzemeID INT PRIMARY KEY,
SponsorID INT,
MalzemeTuru VARCHAR(100),
FOREIGN KEY (SponsorID) REFERENCES Sponsor(SponsorID)
);
```

```
CREATE TABLE LastikTur (
LastikTurID INT PRIMARY KEY,
LastikTuru VARCHAR(100)
);

CREATE TABLE Ilce (
IlceID INT PRIMARY KEY,
IlID INT,
IlceAdi VARCHAR(100),
FOREIGN KEY (IlID) REFERENCES Il(IlID)
);

CREATE TABLE Il (
IlID INT PRIMARY KEY,
IlAdi VARCHAR(100)
);

CREATE TABLE Sponsor (
SponsorID INT PRIMARY KEY,
IlID INT,
SponsorYoneticiID INT,
FOREIGN KEY (IlID) REFERENCES Il(IlID),
FOREIGN KEY (SponsorYoneticiID) REFERENCES SponsorYonetici(SponsorYoneticiID)
);

CREATE TABLE SponsorYonetici (
SponsorYoneticiID INT PRIMARY KEY,
YoneticiAdi VARCHAR(100),
YoneticiSoyadi VARCHAR(100)
);
CREATE TABLE Takim (
TakimID INT PRIMARY KEY,
SponsorID INT,
FOREIGN KEY (SponsorID) REFERENCES Sponsor(SponsorID)
);

CREATE TABLE Surucu (
SurucuID INT PRIMARY KEY,
TakimID INT,
ArabaID INT,
FOREIGN KEY (TakimID) REFERENCES Takim(TakimID),
FOREIGN KEY (ArabaID) REFERENCES Araba(ArabaID)
);
```

```
CREATE TABLE TakimUyesi (
TakimUyesiID INT PRIMARY KEY,
TakimID INT,
FOREIGN KEY (TakimID) REFERENCES Takim(TakimID) );
CREATE TABLE Yaris (
YarisID INT PRIMARY KEY,
IlID INT,
PistID INT,
HavaDurumuID INT,
LastikTuruID INT,
FOREIGN KEY (IlID) REFERENCES Il(IlID),
FOREIGN KEY (PistID) REFERENCES Pist(PistID),
FOREIGN KEY (HavaDurumuID) REFERENCES HavaDurumu(HavaDurumuID),
FOREIGN KEY (LastikTuruID) REFERENCES LastikTur(LastikTurID)
);

CREATE TABLE Pist (
PistID INT PRIMARY KEY,
IlID INT,
PistAdi VARCHAR(100),
FOREIGN KEY (IlID) REFERENCES Il(IlID)
);

CREATE TABLE HavaDurumu (
HavaDurumuID INT PRIMARY KEY,
HavaDurumu VARCHAR(100)
);
```

———SAKLIYORDAM———

```
CREATE OR REPLACE FUNCTION araba_ekle(pk INT, hp INT, colour VARCHAR, wrck
BOOLEAN) RETURNS VOID AS $$
BEGIN

-- Araba tablosuna yeni bir kayıt ekleme
INSERT INTO "CAR" ("CarID", "HorsePower", "Colour", "Wrecked")
VALUES (pk, hp, colour, wrck);

-- İşlem başarılı mesajı
RAISE NOTICE 'Yeni araba kaydı eklendi. CarID: %', pk;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION il_ekle(p1 INT, p2 VARCHAR) RETURNS VOID AS $$
BEGIN

-- Şehir tablosuna yeni bir kayıt ekleme
INSERT INTO "CITY" ("CityID", "CityName")
VALUES (p1, p2);
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION lastik_ekle(pk INT, tType VARCHAR) RETURNS VOID AS $$
BEGIN

-- PREFERRED_TIRE tablosuna yeni bir kayıt ekleme
INSERT INTO "PREFERRED_TIRE" ("TireID", "TireType")
VALUES (pk, tType);

-- İşlem başarılı mesajı
RAISE NOTICE 'Yeni lastik kaydı eklendi. TireID: %', pk;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION pist_ekle(p1 INT, p2 INT, p3 INT, p4 INT, p5 INT) RETURNS
VOID AS $$
BEGIN

-- "TRACK" tablosuna yeni bir kayıt ekleme
INSERT INTO "TRACK" ("TrackID", "CityID", "Distance", "TurnsCount", "TrackCapacity")
VALUES (p1, p2, p3, p4, p5);
END;
$$ LANGUAGE plpgsql;
```

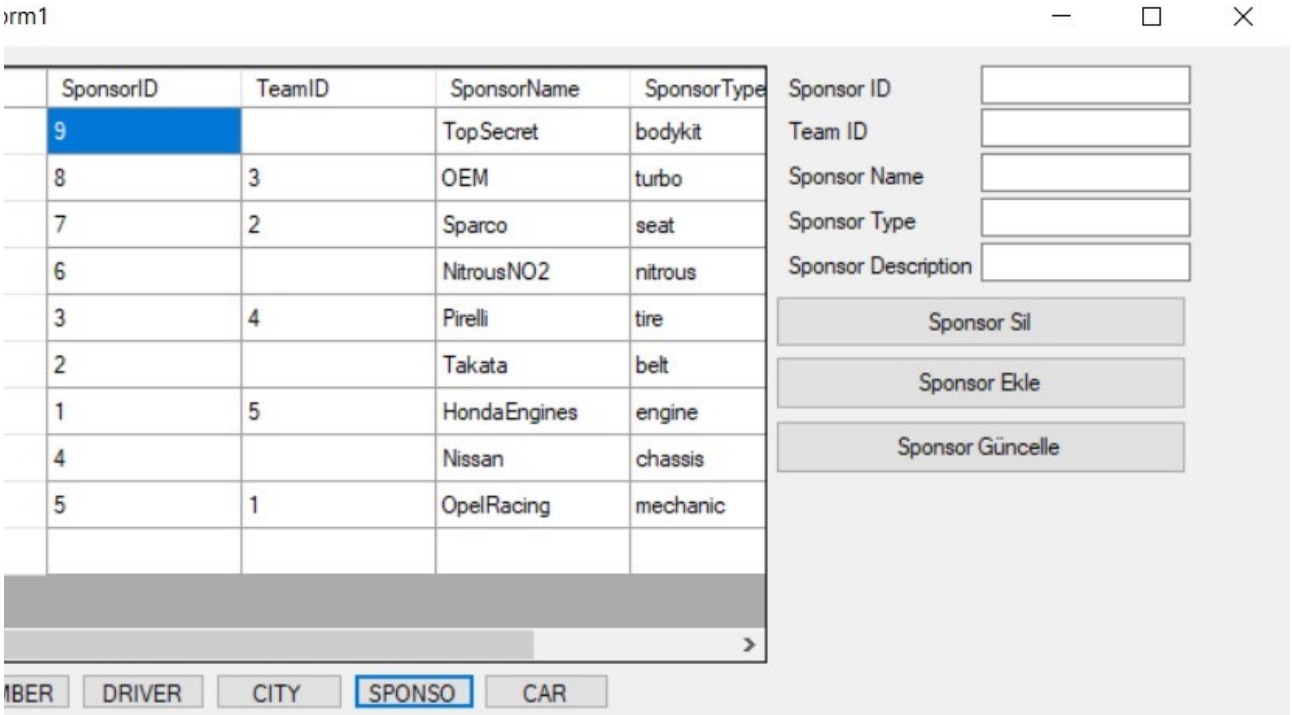
———TRIGGERS———

```
CREATE OR REPLACE FUNCTION LastikTuruEklemeTetikleyici()
RETURNS TRIGGER AS $$
BEGIN

-- Yeni bir lastik türü eklemeden önce mevcut kontrolü yapılır.
IF EXISTS (SELECT 1 FROM "PREFERRED_TIRE" WHERE "TireType" = NEW."TireType") THEN
RAISE EXCEPTION 'Bu lastik türü zaten mevcut.';
END IF;
```

```
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Lastik türü ekleme işlemi yapılacak tablo üzerinde çalışacak tetikleyici
CREATE TRIGGER LastikTuruEklemeKontrolTetikleyici
BEFORE INSERT ON "PREFERRED_TIRE"
FOR EACH ROWXECUTE FUNCTION LastikTuruEklemeTetikleyici();

CREATE OR REPLACE FUNCTION YarispistiEklemeTetikleyici()
RETURNS TRIGGER AS $$
BEGIN

-- Yarış pisti eklemeden önce geçerlilik kontrolü yapılır.
IF NOT EXISTS (SELECT 1 FROM "TRACK" WHERE "TrackID" = NEW."TrackID") THEN
RAISE EXCEPTION 'Geçersiz pist IDsi, lütfen mevcut bir pist belirtin.';
END IF;

IF NOT EXISTS (SELECT 1 FROM "CITY" WHERE "CityID" = NEW."IlID") THEN
RAISE EXCEPTION 'Bu pistin bulunduğu il mevcut değil.';
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Yarış pisti ekleme işlemi yapılacak tablo üzerinde çalışacak tetikleyici
CREATE TRIGGER YarispistiEklemeKontrolTetikleyici
BEFORE INSERT ON "TRACK"
FOR EACH ROW
EXECUTE FUNCTION YarispistiEklemeTetikleyici();

CREATE OR REPLACE FUNCTION SponsorMalzemeEklemeTetikleyici()
RETURNS TRIGGER AS $$
BEGIN

-- Yeni bir malzeme eklenirken veya sponsoru değiştirilirken kontrol yapılır.
IF NEW."SponsorID" IS NULL OR NOT EXISTS (SELECT 1 FROM "SPONSOR" WHERE
"SponsorID" = NEW."SponsorID") THEN
RAISE EXCEPTION 'Geçersiz sponsor, lütfen geçerli bir sponsor belirtin.';
END IF;
```

```
-- Eğer geçerli bir sponsor belirtilmişse işlem devam eder ve yeni malzeme eklenir.
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Malzeme ekleme işlemi yapılacak tablo üzerinde çalışacak tetikleyici
CREATE TRIGGER MalzemeEklemeTetikleyici
BEFORE INSERT OR UPDATE ON "PARTS"
FOR EACH ROW
EXECUTE FUNCTION SponsorMalzemeEklemeTetikleyici();
```

— — —C# ARAYÜZ— — —



https://github.com/m-salihtysz/VeriTaban-Proje